

COLLEGE CODERS

React JS Lesson 24 – useMemo Performance Optimization (Premium Notes)

1) What is an Expensive Calculation

- An expensive calculation is a heavy operation that takes noticeable time to compute.
- Examples include large loops, filtering big datasets, and complex math.

Example:

```
for(let i=0;i<1000000000;i++){  
  // heavy loop  
}
```

Explanation:

- Large loops take longer time and can block UI temporarily.

2) Why Heavy Calculations Cause Slow Re-renders

- When React re-renders, all calculations inside component run again.
- Heavy calculations make UI slow.

Example:

```
const result = heavyCalculation(number);
```

Explanation:

- Every render runs calculation again if not optimized.

3) Practical Example (Without useMemo)

- Calculation runs every render.

Example:

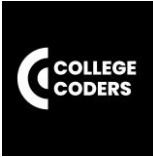
```
const value = slowFunction(num);
```

Explanation:

- Even unrelated state updates trigger this calculation again.

4) Performance Issue Demonstration

- Console logs show repeated heavy calculations.



COLLEGE CODERS

- UI may freeze briefly.

Example:

```
console.log('Heavy Calculation Running');
```

Explanation:

- Console helps observe unnecessary recalculations.

5) Introduction to useMemo

- useMemo stores computed value and reuses it unless dependencies change.

Example:

```
const memoValue = useMemo(()=>{  
  return heavyCalculation(num);  
},[num]);
```

Explanation:

- Computation runs only when dependency changes.

6) Syntax and Dependency Array

- useMemo takes a function and dependency array.

Example:

```
useMemo(()=>{  
  // calculation  
},[dependency])
```

Explanation:

- Dependency decides when recalculation happens.

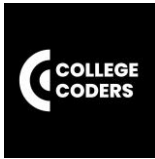
7) Practical Example (With useMemo Optimization)

- Calculation runs only when needed.

Example:

```
const result = useMemo(()=> slowFunction(num), [num]);
```

Explanation:



COLLEGE CODERS

- Re-render won't recompute unless num changes.

8) Real Example (Filtering / Calculation Use Case)

- Commonly used for filtering lists or computing totals.

Example:

```
const filtered = useMemo(()=>{  
  return products.filter(p=>p.price > 100);  
},[products]);
```

Explanation:

- Filtering runs only when product list changes.

9) When to Use useMemo

- Use when expensive calculations slow UI.
- Useful with large datasets or complex logic.

Example:

Large list filtering

Explanation:

- Helps avoid repeated heavy computations.

10) When NOT to Use useMemo

- Avoid for simple calculations.
- useMemo itself adds overhead.

Example:

```
const total = a + b
```

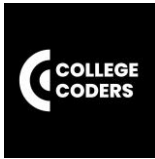
Explanation:

- Simple operations don't need memoization.

Mini Project – Expensive Counter Calculation

Counter updates + heavy calculation optimized using useMemo

- Create counter state.
- Add heavy calculation function.



COLLEGE CODERS

- Optimize result using useMemo.